Wei Wu

2825 Bluegrass Ln, Henderson, NV · 608-960-3382 · weiwu3@alumni.cmu.edu · weiwu3.github.io · linkedin.com/in/weiwu3 · github.com/weiwu3

SKILLS

Languages: Python, C++, C#, JavaScript, SQL, HTML/CSS, Bash
Backend & Web: FastAPI, Flask (Uvicorn/Gunicorn), Socket.IO, REST APIs, asyncio (aiohttp/httpx), SQLModel/SQLAlchemy, Auth0 / OAuth
Frontend & Graphics: Vanilla-JS single-page apps, CanvasKit/Skia, WebAssembly (Emscripten/Rust), Vue.js, Bootstrap 5
Computer Vision: OpenCV, NumPy, SciPy, scikit-image, Shapely, trimesh/Open3D, ChArUco/homography, sub-pixel detection, dithering
AI / ML: PyTorch, CUDA, TensorRT, HuggingFace Transformers, Segment-Anything (SAM2), Depth-Anything-V2, BiRefNet
Data & Cloud: MySQL, SQLite, Redis, Azure (IoT Hub, Blob Storage, Key Vault)
DevOps & Infra: Docker, Portainer, Gitea, TeamCity, Traefik, nginx, systemd, FRP, Paramiko, Selenium, PyInstaller
Embedded & Real-time: Raspberry Pi CM4/CM5, RP2040/PIO firmware, galvo control, Picamera2, pyserial/I2C/GPIO

EDUCATION

Carnegie Mellon University May 2023 | GPA 3.93/4.0
M.S., Mechanical Engineering & Engineering Technology Innovation Management
Coursework: Computer Vision (OpenCV), Engineering Computation (C++), Linear Control (Python)

University of Wisconsin–Madison May 2021 | GPA 3.72/4.0
B.S., Mechanical Engineering; Certificate in Entrepreneurship (Dean's List, 6 semesters)

EXPERIENCE

Full Spectrum Laser — Las Vegas, NV · *Full Stack Software Engineer (Backend)* July 2023 – Present

- Built and solely own the **CPU-offload service** through three architectures — an asyncio job server, then a process-per-request design, then the current **auto-scaling pool** of reusable worker processes draining a shared queue, self-sized from host CPU and RAM, with 50 ms memory monitoring that kills and respawns any worker over budget. **35K jobs over 6 months at 205 ms p50.**
- Designed and own the **compiler** turning user design files (SVG, DXF, PDF, images, 3D point clouds) into the binary instructions the device firmware executes — geometry transforms, rasterization, dithering and infilling, and config models for **30+ machine variants** — plus the optimization passes on top: a **Traveling-Salesman toolpath optimizer** to cut travel time and auto-nesting to reduce material waste.
- Rewrote the platform backend from **synchronous Flask to async FastAPI + Socket.IO**, **serving 500+ registered users** — top contributor to both the outgoing Flask app and the FastAPI service that replaced it, and **led the 39-file blocking-to-async refactor** of its Azure I/O. Re-architected the real-time layer from a 3,100-line Socket.IO monolith into domain-scoped event handlers; contributed across the MySQL/SQLModel data layer and Auth0 OAuth integration.
- Built the **multi-server in-memory state cache** in front of MySQL — read-through with **write-through and dirty checking**, so a device heartbeating twice a second with unchanged state costs **zero DB writes (~170K/day avoided per machine)**; asyncio-locked, invalidated across servers over HTTP, and caching per-device signing keys so job submission skips an Azure IoT round-trip.
- Designed the **headless Jobs REST API** — job generation, submission, control and status over HTTP — with **SHA-256 content-addressed caching** so identical uploads skip re-upload, and tiered device authentication: same-network callers pass on IP, remote callers must enter the TOTP code displayed on the machine's touchscreen. **40K+ requests/month** from an OEM integration.
- Owned **containerization, CI/CD and fleet delivery** — authored **every Dockerfile and build script across seven repos** on a self-hosted Docker / Gitea-registry / Portainer pipeline. Built the internal fleet-deployment console used to push releases across the device fleet (per-device package sync to a desired state, per-package pin and skip, live online/offline status). On the Azure IoT over-the-air update system, authored device bootstrap, A/B slot provisioning, and a **safety gate that fails closed on untagged hardware** to prevent accidental updates to customer machines.
- Solely built the **offline-server appliance** — hardened headless Ubuntu with **LUKS + TPM2 auto-unlock sealed to Secure Boot state**, configured as the site router and bridged WiFi AP (dnsmasq, hostapd, NAT, nginx), and wrote the **offline implementations of the Azure blob and IoT-query layers** so the full stack runs air-gapped; shipped per-customer units to B2B clients.
- **Largest contributor to the laser-scanner calibration and computer-vision pipeline** — ChArUco and grid detection with **sub-pixel accuracy**, a robust RANSAC homography mapping 225 grid points from hardware units to real-world millimeters, multi-frame stitching, and the operator-facing calibration workspace. **Co-developed a predictive calibration model** that replaced a dense per-tile scan with a sparse sample set — **cutting capture time ~6x and improving placement accuracy to 20 µm over 10 mm (0.2%)**, cross-validated with a dedicated regression suite.
- Built the **image-to-3D engraving pipeline** end to end — GPU inference servers I wrote and run (depth-map generation, background removal, image segmentation), the point-cloud generation that turns a depth map into laser fire positions inside glass, and an in-browser vanilla-JS engine that moved that generation off the GPU server, with a **hand-written 16-bit depth-map decoder**. **Independently built and deployed image.fslaser.com** on top of it (Flask, RQ job queue, Azure Blob), replacing paid third-party APIs with self-hosted models.
- Optimized **on-device motion-control streaming** for a **25 kHz** galvo pipeline on Raspberry Pi CM4 — replaced a per-point Python generator with a **vectorized NumPy producer** to eliminate buffer underruns on dense curves (2.6-3.9x faster per vertex, bit-exact), and added a watchdog that halts a job when the stream starves while the laser is firing.
- Earlier (2023): added **3D engraving inside glass and crystal to RE3**, the **C# / .NET Blazor** application serving the machine's local network interface — authored 72% of its 3D user interface — plus the Flask AI backend service behind it.

MathWorks — Natick, MA · *Autonomous Driving Product Marketing Intern* May – Aug 2022

- Ran competitive analysis of RoadRunner vs. rival ADAS simulation tools; findings directly shaped the 2-year product roadmap.
- Relayed customer feature requests to engineering; gained familiarity with the ADAS simulation stack (sensor fusion, scenario generation, HD maps).

PROJECTS

- **ADAS Semantic Segmentation** — *Carnegie Mellon University, 2022* — Improved a VGG16-UNet model from 73% → 86% accuracy with shadow-mitigation pre-processing and OpenCV contour-correction post-processing.
- **UAV Flight Simulator** — *Carnegie Mellon University, 2021* — Built a drone flight simulator in C++/OpenGL with OOP-modeled aerodynamics, automatic obstacle avoidance, and a real-time interactive GUI. github.com/afiretony/24780-Engineers-Republic